

## Department of Computer Science and Engineering

### Individual Contribution Report — Team Assignment

To be completed by EACH student individually and submitted along with the team's assignment. This report explains what YOU specifically did, so it must be written in your own words and reflect only your own work.

#### A. Assignment & Team Information

|                          |                                                                                             |
|--------------------------|---------------------------------------------------------------------------------------------|
| Course Code & Name       | CSA0620 – Design and Analysis of Algorithms                                                 |
| Assignment Title         | Efficient Nearest-Neighbour Detection for Air-Traffic Collision Alerts (Divide-and-Conquer) |
| Team / Group No.         | 4                                                                                           |
| Team Members (all names) | R SARVESH 192521143<br>PRITHIYANKA SHREE M 192521144<br>BEFFY A SHANIKA 192511387           |
| Date of Submission       | 03/09/2026                                                                                  |

#### B. Student's Own Details

|                     |                                                                                                                                                       |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| Student Name        | R SARVESH                                                                                                                                             |
| Register / Roll No. | 192521143                                                                                                                                             |
| Role in the Team    | Implementer — implemented, tested, and debugged the Divide-and-Conquer nearest-neighbour detection module for efficient air-traffic collision alerts. |

#### C. Task-wise Individual Contribution

List every task/module you personally worked on. Add rows if needed. Do not describe work done by teammates.

| S.No | Task / Module Handled                          | Description of Work Done (in your own words)                                                                                                                                                                                                      | Tools / Techniques Used      | Approx. Hours Spent |
|------|------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------|---------------------|
| 1    | Problem understanding & formulation            | Analysed the SkyGuard air-traffic problem statement, identified the closest-pair-of-points formulation, listed the assumptions (2-D Euclidean distance, single-sweep static positions) and constraints (real-time, correctness, resource limits). | MS Word, manual analysis     | 2 hrs               |
| 2    | Brute-Force algorithm & hand trace             | Designed the $O(n^2)$ Brute-Force pseudocode and hand-computed all 21 pairwise Euclidean distances for the Annexure A dataset (P1–P7) to establish the correctness baseline and identify the closest pair (P1–P6).                                | Manual calculation, MS Excel | 3 hrs               |
| 3    | Divide-and-Conquer algorithm & recursion trace | Designed the Divide-and-Conquer pseudocode (sort, divide, conquer, strip-merge) and manually traced the recursion on the Annexure A dataset, verifying it produced the same closest pair as the Brute-Force method.                               | Manual derivation, MS Word   | 4 hrs               |
| 4    | Python implementation & testing                | Implemented both algorithms in Python, tested them on the Annexure A dataset and a randomized $n = 15$ aircraft dataset, and confirmed both algorithms returned identical closest pairs.                                                          | Python 3, VS Code            | 4 hrs               |
| 5    | Complexity analysis (Part C)                   | Derived the time complexity of both algorithms ( $O(n^2)$ vs $O(n \log n)$ ) using the recurrence relation and the Master Theorem, and analysed space complexity and constant-factor overheads.                                                   | Manual derivation            | 2 hrs               |

|   |                                              |                                                                                                                                                                                  |                    |       |
|---|----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------|-------|
| 6 | Experimental evaluation & performance charts | Ran a scalability experiment on synthetic datasets from $n = 10$ to $n = 6,400$ , recorded execution time and comparison counts, and plotted the comparative performance charts. | Python, Matplotlib | 3 hrs |
| 7 | Report documentation & compilation           | Compiled the problem understanding, algorithm design, results, complexity analysis, broader considerations, conclusion and reflection sections into the final assignment report. | MS Word            | 3 hrs |

## D. Team Contribution Percentage Summary

To be filled identically by every team member so contributions are consistent and sum to 100%. Disagreements should be discussed and resolved before submission.

| Team Member Name    | Register No. | % Contribution | Signature           |
|---------------------|--------------|----------------|---------------------|
| R SARVESH           | 192521143    | 33.3333%       | R SARVESH           |
| PRITHIYANKA SHREE M | 192521144    | 33.3333%       | PRITHYANAKA SHREE M |
| BEFFY A SHANIKA     | 192511387    | 33.3333%       | BEFFY A SHANIKA     |
| <b>Total</b>        |              | <b>100%</b>    |                     |

## E. Course Outcomes / Skills Applied

| Course Outcome(s) Addressed by MY Work                           | How I Applied It                                                                                                                                  |
|------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| CO1 – Asymptotic analysis of algorithms using Big-O notation     | Derived and compared the $O(n^2)$ and $O(n \log n)$ time complexities of the two algorithms using the recurrence relation and the Master Theorem. |
| CO2 – Brute-force problem-solving techniques                     | Designed and hand-traced the exhaustive pairwise-comparison Brute-Force algorithm on the Annexure A dataset.                                      |
| CO3 – Divide-and-conquer algorithm design for geometric problems | Designed, traced and implemented the Divide-and-Conquer closest-pair algorithm, including the strip-merge step.                                   |

## F. Challenges Faced and How I Resolved Them

Describe any technical or teamwork challenges you personally faced, and the steps you took to resolve them.

One challenge I faced was correctly implementing the strip-merge step of the Divide-and-Conquer algorithm — my first version compared every point in the strip against every other point, which accidentally degraded performance back toward  $O(n^2)$  for large inputs. I resolved this by re-reading the geometric packing argument (only points within a  $d \times 2d$  rectangle can be closer than  $d$ ), which showed that each point only needs to be checked against the next few points in the y-sorted strip; limiting the inner loop this way restored the expected  $O(n \log n)$  behaviour.

A second challenge was verifying correctness on very small recursive sub-problems ( $n \leq 3$ ), where naive base-case handling occasionally mis-split points with identical x-coordinates. I resolved this by hand-tracing the Annexure A dataset step by step and comparing every intermediate result against the Brute-Force baseline until the two algorithms agreed exactly.

## G. Individual Learning Outcome / Reflection

What did YOU personally learn from completing your part of this assignment? How did it build on classroom concepts?

Working through the recursion trace by hand made the strip-merge argument concrete in a way that reading the algorithm's description in class did not — seeing why only a constant number of neighbours need to be checked in the strip tied the geometry directly to the  $O(n)$  merge step. Running the actual timing experiment also taught me that asymptotic complexity is not the whole story: constant factors and recursive overhead matter for small  $n$ , which Big-O notation alone does not capture. This built directly on the classroom coverage of recurrence relations and the Master Theorem by giving me a concrete algorithm to derive and verify the recurrence for myself.

## H. Declaration

I declare that the contribution described above accurately represents the work I personally performed for this team assignment, and that I have not misrepresented the work of any other team member as my own.

| Student Signature | Date       |
|-------------------|------------|
| R SARVESH         | 03/09/2026 |